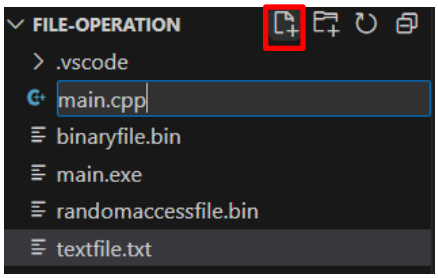
Tutorial 2
Advance File Operation using VS Code

Overview

- This tutorial will demonstrate how to handle file operations in C++ using the <fstream> library. It covers three main types of file operations: text files, binary files, and random access files using Visual Studio Code.

Create a folder and main.cpp

1. Create a new folder
2. Open VS Code. Then click 'Open Folder' and select the folder you have created.
3. Make sure there are 3 files in the folder – textfile.txt, binaryfile.bin and randomaccessfile.bin. These files can be downloaded from the Elearning. Change the content of textfile.txt by **replace with your name.**
4. Next, add file on the folder by selecting the 'New file' button and name the new file as 'main.cpp'.



5. Then write down the following codes:

```
#include <iostream>
#include <fstream>
#include <string>

// Function to open a file with specified mode (text, binary, or random access)
bool openFile(std::fstream &file, const std::string &filename, std::ios_base::openmode mode) {
    file.open(filename, mode);
    if (!file) {
        std::cerr << "Error opening file: " << filename << std::endl;
        return false;
    }
    return true;
}

// Function to write to the file (for text files)
void writeToTextFile(std::fstream &file, const std::string &content) {
    if (file) {
        file << content << std::endl;
    } else {
        std::cerr << "File not open for writing (text)!" << std::endl;
    }
}
```

```
// Function to read from a text file
void readFromTextFile(std::fstream &file) {
    if (file) {
        file.seekg(0, std::ios::beg); // Move the file pointer to the beginning
        std::string line;
        while (std::getline(file, line)) {
            std::cout << line << std::endl;
        }
    } else {
        std::cerr << "File not open for reading (text)!" << std::endl;
    }
}

// Function to write to a binary file
void writeToBinaryFile(std::fstream &file, int number) {
    if (file) {
        file.write(reinterpret_cast<const char*>(&number), sizeof(number));
    } else {
        std::cerr << "File not open for writing (binary)!" << std::endl;
    }
}

// Function to read from a binary file
void readFromBinaryFile(std::fstream &file) {
    if (file) {
        file.seekg(0, std::ios::beg); // Move file pointer to the beginning
        int number;
        while (file.read(reinterpret_cast<char*>(&number), sizeof(number))) {
            std::cout << "Read number: " << number << std::endl;
        }
    } else {
        std::cerr << "File not open for reading (binary)!" << std::endl;
    }
}

// Function for random access: write at a specific position
void writeToRandomAccessFile(std::fstream &file, int position, int value) {
    if (file) {
        file.seekp(position * sizeof(value), std::ios::beg);
        file.write(reinterpret_cast<const char*>(&value), sizeof(value));
    } else {
        std::cerr << "File not open for random access writing!" << std::endl;
    }
}

// Function for random access: read from a specific position
void readFromRandomAccessFile(std::fstream &file, int position) {
    if (file) {
        int value;
        file.seekg(position * sizeof(value), std::ios::beg);
        if (file.read(reinterpret_cast<char*>(&value), sizeof(value))) {
            std::cout << "Read value at position " << position << ": " << value << std::endl;
        } else {
            std::cerr << "Error reading at position " << position << std::endl;
        }
    } else {
        std::cerr << "File not open for random access reading!" << std::endl;
    }
}

// Function to close the file
void closeFile(std::fstream &file) {
    if (file) {
        file.close();
    }
}

int main() {
    std::fstream textFile, binaryFile, randomAccessFile;
    std::string textFilename = "textfile.txt";
    std::string binaryFilename = "binaryfile.bin";
    std::string randomAccessFilename = "randomaccessfile.bin";

    // Open and write to a text file
    if (openFile(textFile, textFilename, std::ios::out | std::ios::app)) {
        writeToTextFile(textFile, "I love coding.");
        closeFile(textFile);
    }

    // Open and read from a text file
    if (openFile(textFile, textFilename, std::ios::in)) {
        std::cout << "\nReading from text file:\n";
        readFromTextFile(textFile);
        closeFile(textFile);
    }
```

```
    // Open and write to a binary file
    if (openFile(binaryFile, binaryFilename, std::ios::out | std::ios::binary | std::ios::app)) {
        writeToBinaryFile(binaryFile, 123);
        writeToBinaryFile(binaryFile, 456);
        closeFile(binaryFile);
    }

    // Open and read from a binary file
    if (openFile(binaryFile, binaryFilename, std::ios::in | std::ios::binary)) {
        std::cout << "\nReading from binary file:\n";
        readFromBinaryFile(binaryFile);
        closeFile(binaryFile);
    }

    // Open and write to a random access file
    if (openFile(randomAccessFile, randomAccessFilename, std::ios::in | std::ios::out | std::ios::binary | std::ios::app)) {
        writeToRandomAccessFile(randomAccessFile, 0, 1000); // Write 1000 at position 0
        writeToRandomAccessFile(randomAccessFile, 1, 2000); // Write 2000 at position 1
        closeFile(randomAccessFile);
    }

    // Open and read from a specific position in the random access file
    if (openFile(randomAccessFile, randomAccessFilename, std::ios::in | std::ios::binary)) {
        std::cout << "\nReading from random access file:\n";
        readFromRandomAccessFile(randomAccessFile, 0); // Read value at position 0
        readFromRandomAccessFile(randomAccessFile, 1); // Read value at position 1
        closeFile(randomAccessFile);
    }

    return 0;
}
```

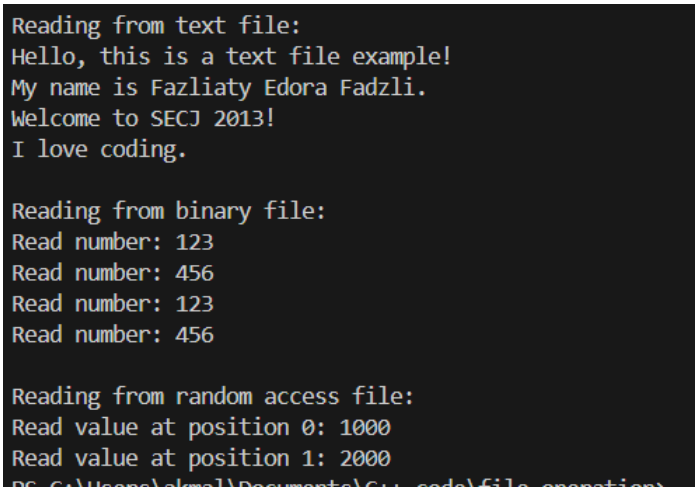
Explanation

1. Here's a brief overview of the tutorial:

    - Opening Files:
        - The *openFile* function opens a file in specified modes (text, binary, or random access) and checks for success.
    - Text File Operations:
        - *writeToTextFile* writes a string to a text file.
        - *readFromTextFile* reads and displays the contents of the text file line by line.
    - Binary File Operations:
        - *writeToBinaryFile* writes an integer to a binary file.
        - *readFromBinaryFile* reads and displays integers from the binary file.
    - Random Access File Operations:
        - *writeToRandomAccessFile* writes integers at specific positions in a binary file.
        - *readFromRandomAccessFile* reads integers from specified positions.
    - Closing Files: The *closeFile* function ensures files are closed properly to prevent resource leaks.
    - Main Function: The main function coordinates the various file operations, demonstrating how to handle text and binary files, including random access.

Compile and Run

2. Press F5 to run the code.
3. The output will show the:



4. Upload all outputs in Elearning Forum. Reply to main thread created in the forum your name, matric no & the output image.